SKYNETIX SYSTEMS

AGENTIC AI
HANDS-ON LAB WORKBOOK

120 practical labs aligned chapter-by-chapter with the Agentic AI Training Handbook

---

Learn by building. Code every concept. Demonstrate every result.

Student edition with instructor checkpoints, deliverables, stretch tasks and selected reference solutions.

Version 1.0 | November 2026

Prepared for SkyNetix Systems training classes

How to use this workbook

This workbook follows the same 30-chapter sequence as the Agentic AI Training Handbook. Each chapter contains four labs, giving learners 120 practical tasks. Complete the labs in order: later chapters assume that the learner can write basic Python, use functions, handle JSON and call APIs.

Recommended class rhythm: Explain the objective (5–10 minutes), demonstrate only the first step, let students code (20–40 minutes), review output and errors, then assign the stretch task.

Submission rule: For every lab, submit source code, sample input, sample output and a short explanation of one issue encountered and how it was fixed.

Safety: Use synthetic data. Do not paste company secrets, customer records, passwords or private documents into public AI services. Never run untrusted code or give an agent unrestricted access to external systems.

Environment setup checklist

Setup 1. Install a current stable Python 3 version and verify it with **python --version**.

Setup 2. Install VS Code or another editor and enable Python support.

Setup 3. Create a project folder named **agentic-ai-labs**.

Setup 4. Create and activate a virtual environment named **.venv**.

Setup 5. Create a **requirements.txt** file and install dependencies inside the environment.

Setup 6. Create a **.gitignore** file that excludes .venv, .env, __pycache__ and generated secrets.

```
python -m venv .venv
# Windows PowerShell
.venv\\Scripts\\Activate.ps1
# macOS / Linux
source .venv/bin/activate
python -m pip install --upgrade pip
```

Progress tracker

Part	Chapters	Labs	Completed
Python foundation	1–9	36	■■■■
AI and LLM foundation	10–15	24	■■■■
Agent engineering	16–21	24	■■■■
RAG and integrations	22–26	20	■■■■
Production and capstone	27–30	16	■■■■

Chapter-by-chapter lab map

Each chapter below has four practical labs. The expected deliverable is listed so the instructor can assess completion consistently.

1. What is AI, Generative AI and Agentic AI? — 4 labs

2. Python setup and first program — 4 labs

3. Python variables, types and operators — 4 labs

4. Conditions, loops and control flow — 4 labs

5. Collections: lists, tuples, sets and dictionaries — 4 labs

6. Functions, modules and error handling — 4 labs

7. Files, JSON and working with data — 4 labs

8. Object-oriented Python — 4 labs

9. APIs, HTTP and environment variables — 4 labs

10. Machine learning and neural network essentials — 4 labs

11. Large language models explained — 4 labs

12. Prompt engineering and conversation structure — 4 labs

13. Calling an LLM from Python — 4 labs

14. Structured outputs and validation — 4 labs

15. Function calling and tools — 4 labs

16. The agent loop: reason, act, observe — 4 labs

17. Building a simple tool-using agent — 4 labs

18. Agent memory and state — 4 labs

19. Planning, workflows and human approval — 4 labs

20. LangChain concepts and practical patterns — 4 labs

21. LangGraph: stateful agent workflows — 4 labs

22. Retrieval-augmented generation (RAG) — 4 labs

23. Embeddings and vector databases — 4 labs

24. Multi-agent systems and CrewAI concepts — 4 labs

25. MCP and connecting external tools — 4 labs

26. n8n for AI automation — 4 labs

27. Testing, evaluation and observability — 4 labs

28. Security, privacy and responsible AI — 4 labs

29. Deployment and production checklist — 4 labs

30. Capstone projects, assessments and glossary — 4 labs

1. What is AI, Generative AI and Agentic AI?

Chapter objective: Identify the difference between a fixed program, a generative chatbot and an agent that can use tools.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Build a rule-based FAQ responder with at least five question-and-answer pairs. It must show a friendly fallback when no match is found.

Lab 2. Create a decision table comparing a calculator script, an LLM chatbot and a tool-using agent. Include input, processing, output and limitations.

Lab 3. Draw a simple agent architecture with user, model, tool registry, tools and final response. Label the direction of data flow.

Lab 4. Choose a real-world task such as preparing a meeting summary. Break it into at least five steps and mark which steps need a person, model or tool.

Expected deliverable

A working FAQ script, comparison table, architecture diagram and task decomposition.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add input validation, helpful error messages and at least three tests.

2. Python setup and first program

Chapter objective: Get every learner running Python locally and comfortable editing, saving and executing a script.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create hello_agent.py. Print a welcome message, your name and the course title on separate lines.

Lab 2. Use an f-string to display a learner name, current module and number of labs completed.

Lab 3. Ask the user for a name and preferred AI topic using input(), then print a personalized welcome.

Lab 4. Create a menu-driven program that displays 1. Python, 2. LLMs, 3. Agents and 4. Exit. Handle an invalid menu choice.

Expected deliverable

Four runnable Python files or one menu application with clear terminal output.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Refactor repeated code into a reusable function and document its parameters.

3. Python variables, types and operators

Chapter objective: Represent application data with suitable types and perform basic calculations.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create variables for student name, age, course fee, attendance percentage and enrollment status. Print each value and type.

Lab 2. Build a course-fee calculator that accepts fee and discount percentage and displays the final amount.

Lab 3. Convert a temperature from Celsius to Fahrenheit. Validate the numeric input and display a useful error message.

Lab 4. Create a token-cost estimator using user-entered input-token count, output-token count and illustrative per-token rates. Label the rates as sample values, not provider prices.

Expected deliverable

A small calculator program with numeric conversions, formatted output and validation.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add a small feature requested by another learner and test that existing behavior still works.

4. Conditions, loops and control flow

Chapter objective: Use branching and repetition for routing, retries and repeated processing.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Ask for a score from 0 to 100 and assign a grade using if/elif/else. Reject out-of-range values.

Lab 2. Print multiplication tables from 1 to 10 using nested loops.

Lab 3. Simulate an API call that fails twice and succeeds on the third attempt. Stop immediately after success.

Lab 4. Process a list of support tickets and count open, closed and urgent tickets. Use continue to skip invalid records.

Expected deliverable

Programs demonstrate conditions, for/while loops, break, continue and bounded retries.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Write a README with setup instructions, sample input/output and known limitations.

5. Collections: lists, tuples, sets and dictionaries

Chapter objective: Model tool lists, messages, profiles and structured records.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create a list of eight AI topics. Add two topics, remove one and print the remaining topics alphabetically.

Lab 2. Store a latitude/longitude pair in a tuple and explain why a tuple can be suitable for fixed coordinates.

Lab 3. Create a set from a list containing duplicate course topics. Display the unique values.

Lab 4. Represent three conversation messages as dictionaries with role and content. Print only user messages.

Expected deliverable

A mini data model using list, tuple, set and dictionary correctly.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add input validation, helpful error messages and at least three tests.

6. Functions, modules and error handling

Chapter objective: Write reusable functions and handle expected failures without hiding bugs.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Write calculate_total(price, quantity) with type hints and checks for negative values.

Lab 2. Write safe_divide(a, b). Return a clear result or raise a meaningful error when the divisor is zero.

Lab 3. Create a function that accepts a list of scores and returns average, minimum and maximum. Handle an empty list.

Lab 4. Move at least three reusable functions into a module named agent_utils.py and import them from app.py.

Expected deliverable

A modular Python project with functions, type hints and deliberate exception handling.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Refactor repeated code into a reusable function and document its parameters.

7. Files, JSON and working with data

Chapter objective: Read and write files and exchange structured data using JSON.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create notes.txt, write three learning notes and read them back line by line.

Lab 2. Create learner.json containing name, level, topics and completed labs. Load it and print selected fields.

Lab 3. Read a JSON list of FAQs and search it by a keyword. Return the matching question and answer.

Lab 4. Handle missing files and malformed JSON with clear error messages. Do not silently replace corrupt data.

Expected deliverable

Text and JSON file operations with robust error handling.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add a small feature requested by another learner and test that existing behavior still works.

8. Object-oriented Python

Chapter objective: Represent tools and agents as objects with clear responsibilities.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create a Student class with name, level and completed_labs. Add a method to increment completed labs.

Lab 2. Create a Tool class with name, description and run(input_data). Instantiate two different tools.

Lab 3. Create a CalculatorTool class that extends Tool and overrides run(). Validate supported operations.

Lab 4. Create an Agent class that receives a list of Tool objects through composition and can list available tool names.

Expected deliverable

A small object-oriented tool framework with at least two tool implementations.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Write a README with setup instructions, sample input/output and known limitations.

9. APIs, HTTP and environment variables

Chapter objective: Connect Python to an HTTP API while protecting credentials and handling failures.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Install requests. Call a public JSON API with a timeout and print two selected response fields.

Lab 2. Add handling for connection errors, timeouts and non-success HTTP status codes.

Lab 3. Read a configurable API base URL from an environment variable and use a safe default for local practice.

Lab 4. Create a .env file with a dummy value, load it using python-dotenv and confirm .env is excluded by .gitignore.

Expected deliverable

A documented API client with timeout, status validation and externalized configuration.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add input validation, helpful error messages and at least three tests.

10. Machine learning and neural network essentials

Chapter objective: Understand the basic ML workflow and distinguish training from inference.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create a tiny labelled dataset of study hours and pass/fail outcomes. Identify features and labels.

Lab 2. Split a sample dataset into training and test sets conceptually or with scikit-learn. Explain why test data must be held out.

Lab 3. Use a simple scikit-learn classifier on a toy dataset, print predictions and report accuracy. Do not interpret toy accuracy as real-world performance.

Lab 4. Write a one-page model card describing intended use, data limitations, evaluation and likely failure cases.

Expected deliverable

A small ML demonstration plus a clear explanation of training, inference and limitations.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Refactor repeated code into a reusable function and document its parameters.

11. Large language models explained

Chapter objective: Explore tokens, context, model output and limitations without treating generated text as guaranteed fact.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Prepare five prompts of different lengths and compare how the response changes as context grows.

Lab 2. Ask a model the same question with a concise prompt and a detailed prompt. Record differences in accuracy and completeness.

Lab 3. Create a hallucination-check exercise: ask for three factual claims, then verify each against trusted references.

Lab 4. Write a short explanation of context-window limits, token usage and why long chat history may need summarization.

Expected deliverable

A comparison log and fact-check record with sources.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add a small feature requested by another learner and test that existing behavior still works.

12. Prompt engineering and conversation structure

Chapter objective: Create prompts with roles, context, constraints, examples and output requirements.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Write a system instruction for a beginner Python tutor with tone, teaching sequence and safety boundaries.

Lab 2. Create a prompt template with variables {topic}, {learner_level} and {duration}. Generate three different lesson prompts.

Lab 3. Add two few-shot examples that show the required answer format: definition, code, explanation and quiz.

Lab 4. Test a prompt-injection example embedded in a pretend document. Explain why document text must not override trusted instructions.

Expected deliverable

A prompt library with test cases and documented expected output format.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Write a README with setup instructions, sample input/output and known limitations.

13. Calling an LLM from Python

Chapter objective: Make a model request from Python and handle configuration, output and failure cases.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Install the current provider SDK and configure the API key using an environment variable. Never hard-code a real key.

Lab 2. Send a simple request asking the model to explain a Python concept. Print the returned text.

Lab 3. Refactor the request into a function ask_model(prompt) and add exception handling for API failures.

Lab 4. Build a command-line chat loop that accepts multiple user messages and exits when the user types quit. Add a maximum history length.

Expected deliverable

A working command-line model client; SDK syntax and model ID must be checked against current provider documentation.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add input validation, helpful error messages and at least three tests.

14. Structured outputs and validation

Chapter objective: Turn model-generated data into validated application objects.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Define a Pydantic model for a support ticket with ticket_id, category, priority and description.

Lab 2. Validate a valid ticket and print the parsed fields. Then test missing and incorrectly typed fields.

Lab 3. Add constraints: priority must be one of low, medium or high; description must not be empty.

Lab 4. Create a lesson-plan schema with topic, duration and exercises. Validate a sample response and report validation errors.

Expected deliverable

Pydantic schemas with valid and invalid test cases.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Refactor repeated code into a reusable function and document its parameters.

15. Function calling and tools

Chapter objective: Expose narrowly scoped Python functions as tools and validate all arguments.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Implement a read-only FAQ search tool that accepts a query string and returns matching entries.

Lab 2. Create a tool registry dictionary mapping tool names to Python functions. Return a safe error for unknown names.

Lab 3. Add input validation to a calculator tool. Do not use eval() on untrusted expressions; use explicit operations or a safe parser.

Lab 4. Write unit tests for valid input, invalid input, missing arguments and tool exceptions.

Expected deliverable

A tool registry with documented schemas, validation and unit tests.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add a small feature requested by another learner and test that existing behavior still works.

16. The agent loop: reason, act, observe

Chapter objective: Understand and implement a bounded orchestration loop. Keep model decisions separate from tool execution.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Write pseudocode for a model → tool request → tool result → model cycle. Include a final-answer branch.

Lab 2. Implement a deterministic loop that chooses between FAQ search and calculator based on a simulated model decision.

Lab 3. Add maximum-step and maximum-tool-call limits. Prove that the loop stops when the limit is reached.

Lab 4. Simulate a tool failure and make the orchestrator return the error to the next step without crashing.

Expected deliverable

A bounded loop with explicit stop conditions, tool-result messages and failure handling.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Write a README with setup instructions, sample input/output and known limitations.

17. Building a simple tool-using agent

Chapter objective: Combine model selection, tool registry and application-controlled execution.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Build a mini-agent that can answer FAQs using a local dictionary and can calculate simple totals using a safe function.

Lab 2. Separate tool selection from tool execution. Use a simulated decision object before integrating a real model.

Lab 3. Add an LLM tool-selection request using the provider's current documented function-calling protocol. The application executes only allowlisted tools.

Lab 4. Create five end-to-end tests covering FAQ, calculation, unknown request, malformed arguments and tool failure.

Expected deliverable

A working bounded assistant that uses at least two tools and passes end-to-end tests.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add input validation, helpful error messages and at least three tests.

18. Agent memory and state

Chapter objective: Track task progress and distinguish temporary state from durable user memory and reference knowledge.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create a state dictionary for a course-enrollment workflow with status, current_step and approval_required.

Lab 2. Write update_state() that changes one field while preserving the other fields.

Lab 3. Simulate a multi-turn task where the agent remembers the current topic and completed step during the session.

Lab 4. Design a data-retention table: field, purpose, sensitivity, retention period and deletion method. Use fictional data only.

Expected deliverable

A stateful workflow prototype and a privacy-aware data inventory.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Refactor repeated code into a reusable function and document its parameters.

19. Planning, workflows and human approval

Chapter objective: Select the right orchestration pattern and pause before consequential actions.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Draw a sequential workflow for generating a weekly training report from attendance and assessment data.

Lab 2. Build a router function that sends billing, technical and general support questions to different handlers.

Lab 3. Create an approval gate for sending an email. If approval is false, prove that no send function is called.

Lab 4. Compare a fixed workflow and an agent for three scenarios. Justify the simpler and safer option for each.

Expected deliverable

Workflow diagrams, routing code and a tested human approval gate.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add a small feature requested by another learner and test that existing behavior still works.

20. LangChain concepts and practical patterns

Chapter objective: Use framework abstractions for prompts and model composition while understanding the underlying pieces.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Install the current LangChain core package in a fresh virtual environment and record the installed version.

Lab 2. Create a ChatPromptTemplate with system and human messages and invoke it with a topic variable.

Lab 3. Connect a configured chat model using the current provider integration and inspect the returned message.

Lab 4. Create a small pipeline that accepts a topic, builds a prompt and returns model output. Add a fallback for API failure.

Expected deliverable

A small LangChain pipeline; verify package names and APIs against current official docs.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Write a README with setup instructions, sample input/output and known limitations.

21. LangGraph: stateful agent workflows

Chapter objective: Model a workflow as state, nodes, edges and conditional routing.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Define a TypedDict state with question, category and answer fields.

Lab 2. Write classify and respond node functions that read and update the state dictionary.

Lab 3. Create a graph with a conditional route for billing versus general questions using the current LangGraph API.

Lab 4. Add a human-review node for requests that could cause a business action. Test both approval and rejection paths.

Expected deliverable

A compiled state graph with branching and a human-review path.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add input validation, helpful error messages and at least three tests.

22. Retrieval-augmented generation (RAG)

Chapter objective: Build a retrieval pipeline that grounds model responses in supplied documents.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create a folder of five short fictional policy documents. Add document IDs and metadata.

Lab 2. Load the documents and split them into chunks. Preserve source ID and chunk index for each chunk.

Lab 3. Implement a keyword baseline retriever and return the top matching chunks for five test questions.

Lab 4. Pass retrieved context to a model and require it to cite source IDs or say that evidence was not found.

Expected deliverable

A document ingestion and retrieval prototype with source-aware answers.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Refactor repeated code into a reusable function and document its parameters.

23. Embeddings and vector databases

Chapter objective: Understand vector similarity and use semantic retrieval with metadata filtering.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Write a cosine_similarity() function and test identical, orthogonal and opposite vectors.

Lab 2. Generate embeddings for five sample sentences using a currently supported embedding model. Keep the model name configurable.

Lab 3. Store vectors and metadata in a local or hosted vector store supported by your environment. Retrieve the nearest matches for a query.

Lab 4. Add a metadata filter such as department='training'. Test that a user cannot retrieve a document outside their allowed scope.

Expected deliverable

A semantic-search demo with similarity ranking and metadata-based access control.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add a small feature requested by another learner and test that existing behavior still works.

24. Multi-agent systems and CrewAI concepts

Chapter objective: Divide a task into roles and validate outputs between agents.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Design researcher, analyst and writer roles for a small market-research briefing. Define each role's allowed tools.

Lab 2. Write JSON output contracts for each role so handoffs are predictable.

Lab 3. Implement the workflow first as ordinary Python functions. Pass the research result to analysis and analysis to drafting.

Lab 4. Rebuild the same workflow using the current CrewAI APIs. Compare clarity, cost, execution time and failure handling.

Expected deliverable

A multi-role workflow with structured handoffs and a comparison of framework versus plain Python.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Write a README with setup instructions, sample input/output and known limitations.

25. MCP and connecting external tools

Chapter objective: Understand MCP's host/client/server roles and connect to a deliberately limited local demo server.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Draw the MCP host, client and server architecture. Label tools and resources separately.

Lab 2. Review an example MCP server definition and list every capability it exposes before running it.

Lab 3. Create or configure a read-only demo tool and test its input and output contract using an MCP-compatible client.

Lab 4. Write a security review covering server identity, permissions, untrusted outputs, secrets and approval for write actions.

Expected deliverable

An MCP architecture diagram and a reviewed, limited-scope demo integration.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add input validation, helpful error messages and at least three tests.

26. n8n for AI automation

Chapter objective: Create a workflow that connects a trigger, validation, AI classification, routing and human review.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create a manual-trigger workflow that accepts a sample support request and validates required fields.

Lab 2. Add a classification step that labels the request as billing, technical or general. Validate the label against an allowlist.

Lab 3. Route each category to a separate branch and record the selected branch in the execution output.

Lab 4. Add an approval step before sending any external notification. Add an error branch for API or model failure.

Expected deliverable

An n8n workflow export or screenshots showing successful, rejected and failed executions.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Refactor repeated code into a reusable function and document its parameters.

27. Testing, evaluation and observability

Chapter objective: Test tool correctness, workflow behavior, answer quality, safety and cost.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Write unit tests for a calculator tool, FAQ search and input validation. Include edge cases.

Lab 2. Create an evaluation dataset with at least ten user questions, expected tool choices and expected source documents.

Lab 3. Run the same evaluation before and after a prompt or retrieval change. Compare success rate and unsupported answers.

Lab 4. Add structured logs for run ID, tool name, duration, status and error category. Do not log secrets or unnecessary personal data.

Expected deliverable

A test suite, small evaluation report and privacy-aware execution logs.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add a small feature requested by another learner and test that existing behavior still works.

28. Security, privacy and responsible AI

Chapter objective: Put security controls in the tool layer and test malicious or unauthorized inputs.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create a threat model for an agent that can search documents and draft emails. Identify assets, attackers and trust boundaries.

Lab 2. Write an allowlist that permits only approved tool names and approved email domains in a simulated environment.

Lab 3. Create a prompt-injection test document that asks the agent to reveal secrets. Verify that it is treated as untrusted content.

Lab 4. Add a permission check and human approval gate before a simulated delete, payment or outbound message action.

Expected deliverable

A threat model, negative tests, least-privilege design and approval evidence.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Write a README with setup instructions, sample input/output and known limitations.

29. Deployment and production checklist

Chapter objective: Package an agent service with configuration, health checks, logging and bounded execution.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Create a requirements.txt with pinned versions for your demo application and document the Python version.

Lab 2. Create a FastAPI endpoint that accepts a question and returns a validated response from a local demo function.

Lab 3. Add a health endpoint, request timeout, input-size limit and a maximum agent-step setting.

Lab 4. Write a deployment runbook covering secrets, access control, logs, monitoring, rollback and incident response.

Expected deliverable

A locally runnable API service and a production-readiness runbook.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Add input validation, helpful error messages and at least three tests.

30. Capstone projects, assessments and glossary

Chapter objective: Combine the course concepts into a demonstrable, evaluated application.

Before you start: Open the matching chapter in the Agentic AI Training Handbook. Create a separate folder for this chapter's code and save each lab with a clear filename.

Hands-on labs

Lab 1. Capstone A: Build a Python FAQ assistant with JSON knowledge, search, source IDs, tests and a no-answer fallback.

Lab 2. Capstone B: Build a RAG assistant over approved documents with chunking, embeddings, retrieval, citations and access filters.

Lab 3. Capstone C: Build a support agent with FAQ and ticket tools, bounded steps, structured outputs and human approval before sending a draft.

Lab 4. Capstone D: Build an n8n intake automation with classification, routing, error handling, audit-friendly logs and a manual review gate.

Expected deliverable

A working demo, source code, README, architecture diagram, test results and a five-minute presentation.

Instructor checkpoint

Ask the learner to run the program, explain the flow without reading the code line by line, show one successful test and one failure/edge-case test, and explain what they changed after debugging.

Stretch challenge

Refactor repeated code into a reusable function and document its parameters.

Final practical assessment

Learners should complete one capstone independently and demonstrate the system live. A project is not complete merely because it returns a plausible answer; the learner must show tool permissions, failure handling, tests and limitations.

Area	Weight	Evidence
Requirements and user journey	10%	Problem statement, user flow and scope
Python implementation	15%	Readable, modular code and setup instructions
Agent/workflow design	20%	State, tools, routing, bounded execution
Grounding and correctness	15%	Evidence, citations or source IDs; no-answer behavior
Testing and evaluation	15%	Unit tests, end-to-end tests and evaluation cases
Security and safety	15%	Least privilege, validation, approval and secret handling
Demo and explanation	10%	Working demonstration and clear discussion of trade-offs

Student submission checklist

Submit 1. Source code and requirements.txt are included.

Submit 2. README explains setup, configuration and execution.

Submit 3. No real API keys or confidential data are committed.

Submit 4. Architecture diagram and tool descriptions are included.

Submit 5. At least five meaningful tests are included.

Submit 6. Failure behavior and human approval are demonstrated where applicable.

Submit 7. Known limitations and future improvements are documented.

Instructor answer guidance

This is a practical workbook, so many labs have multiple valid solutions. Grade for correct behavior, safe design, understandable code and evidence of testing—not for matching one exact implementation. For framework-specific labs, confirm current package versions and API syntax in official documentation before class.

Important: API names, model identifiers, framework syntax and hosted-service pricing can change. The instructor should verify the current official documentation before distributing environment-specific starter code.

Useful official references

Python tutorial: https://docs.python.org/3/tutorial/

OpenAI API documentation: https://platform.openai.com/docs/

Hugging Face Agents Course: https://huggingface.co/learn/agents-course/

LangChain documentation: https://docs.langchain.com/

LangGraph documentation: https://docs.langchain.com/oss/python/langgraph/

CrewAI documentation: https://docs.crewai.com/

MCP documentation: https://modelcontextprotocol.io/

n8n documentation: https://docs.n8n.io/

Pydantic documentation: https://docs.pydantic.dev/